中山大學
SUN YAT-SEN UNIVERSITY

Lecture 12
Heaps

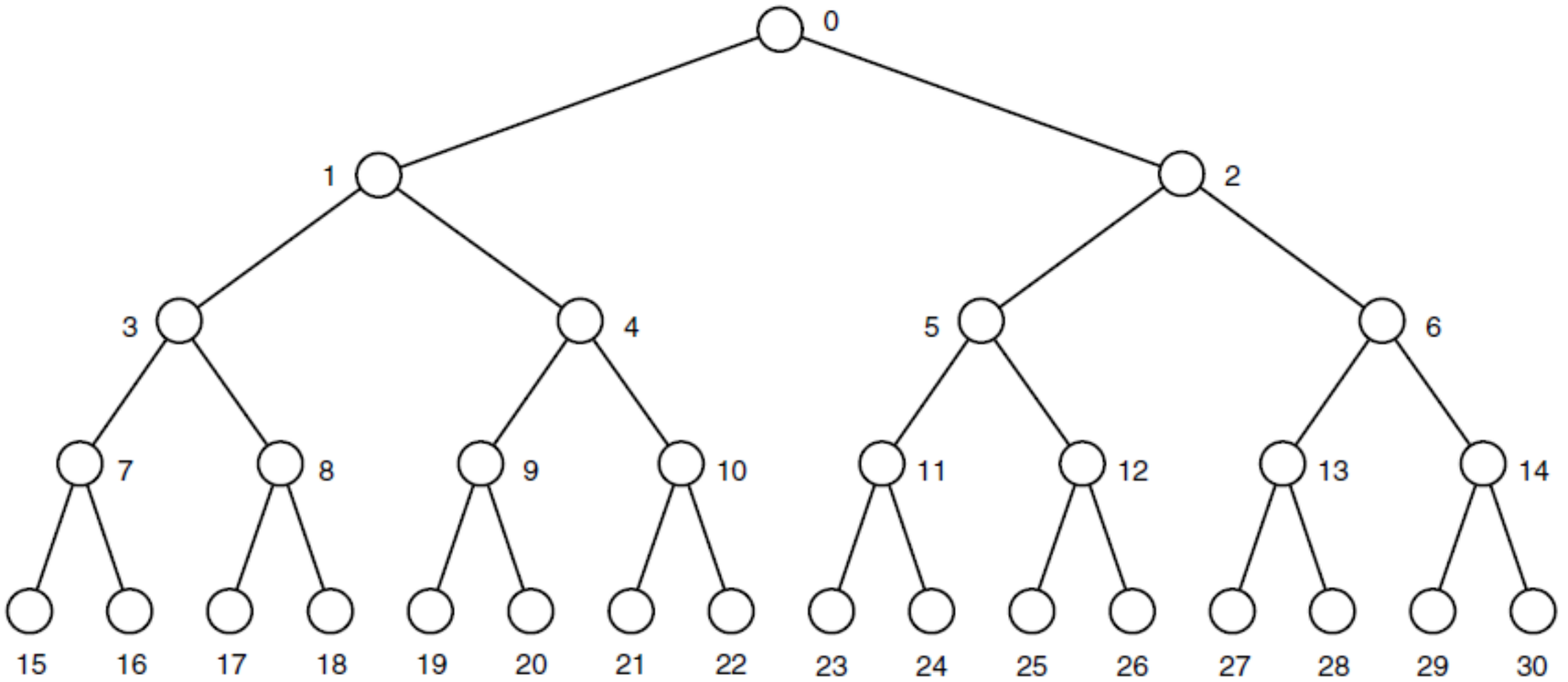


Data Structures and Algorithms

Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

zhangzizhen@gmail.com

Trees, Lists, and Heaps



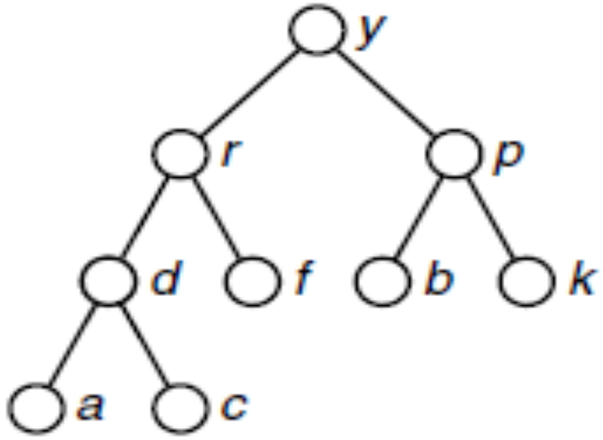
If the root of the tree is in position 0 of the list, then the left and right children of the node in position k are in positions $2k + 1$ and $2k + 2$ of the list, respectively. If these positions are beyond the end of the list, then these children do not exist.

Heaps

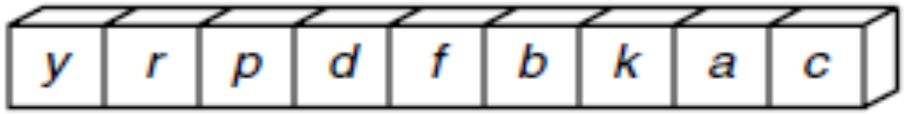
DEFINITION A ***heap*** is a list in which each entry contains a key, and, for all positions k in the list, the key at position k is at least as large as the keys in positions $2k$ and $2k + 1$, provided these positions exist in the list.

- A binary heap is a complete binary tree; that is, all levels of the tree, except possibly the last one (deepest) are fully filled, and, if the last level of the tree is not complete, the nodes of that level are filled from left to right.
- A heap is not an ordered list, there is no necessary ordering between the keys in locations k and k+1(k>1);
- The first entry must have the largest/smallest key in the max-heap/min-heap;
- Heapsort is suitable only for contiguous lists.

A Heap Example



Heap

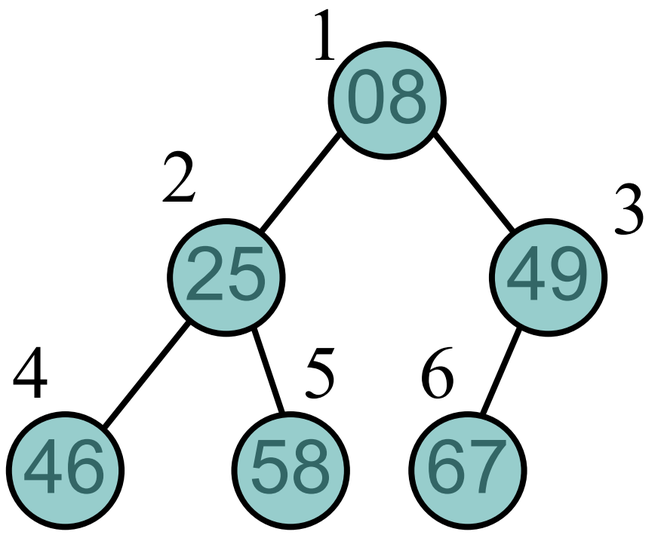


设有n个元素的序列 k_1，k_2，…，k_n，当且仅当满足下述关系之一时，称之为堆。

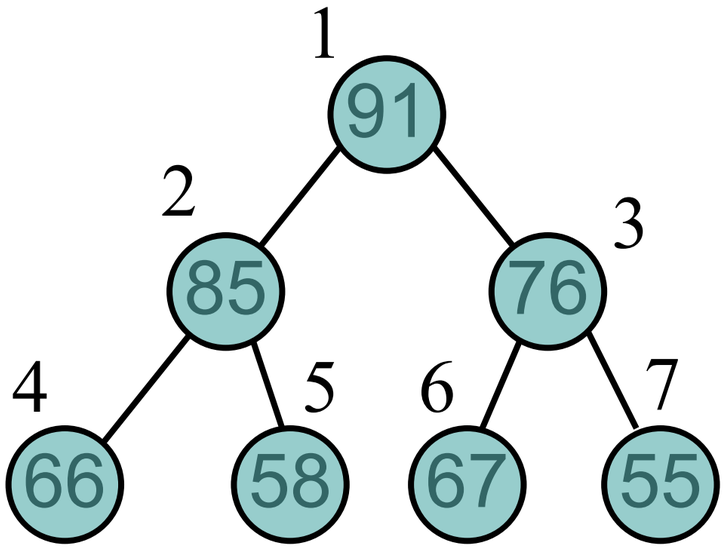
$$\begin{cases} k_i \le k_{2i} \\ k_i \le k_{2i+1} \end{cases} \quad \text{或者} \quad \begin{cases} k_i \ge k_{2i} \\ k_i \ge k_{2i+1} \end{cases} \qquad i=1, 2,\dots\ n/2$$

A Heap Example

有序列T1=（08, 25, 49, 46, 58, 67）和序列T2=（91, 85, 76, 66, 58, 67, 55）, 判断它们是否为“堆”?



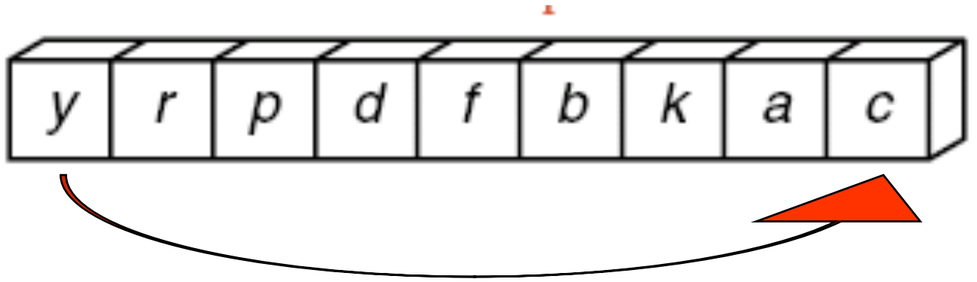
√ （小根堆）



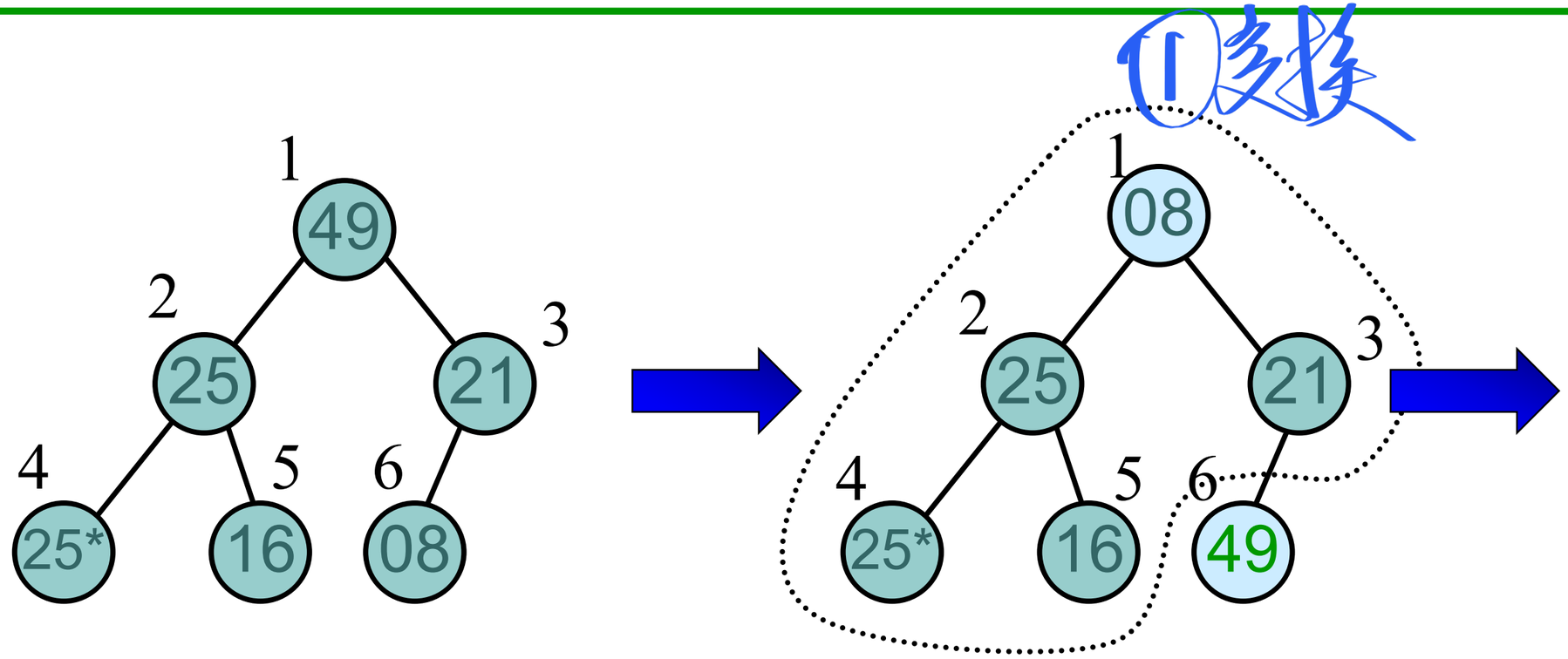
√ （大根堆）

Heapsort Method

- Heapsort proceeds in two phases:
  - Arrange the entries in the list into a heap; (建堆)
  - Repeatedly remove the top of the heap and promote another entry to take its place (restore the heap). (调整堆)
- In the second phase, we move the top of the heap to the last position.



A Heapsort Example



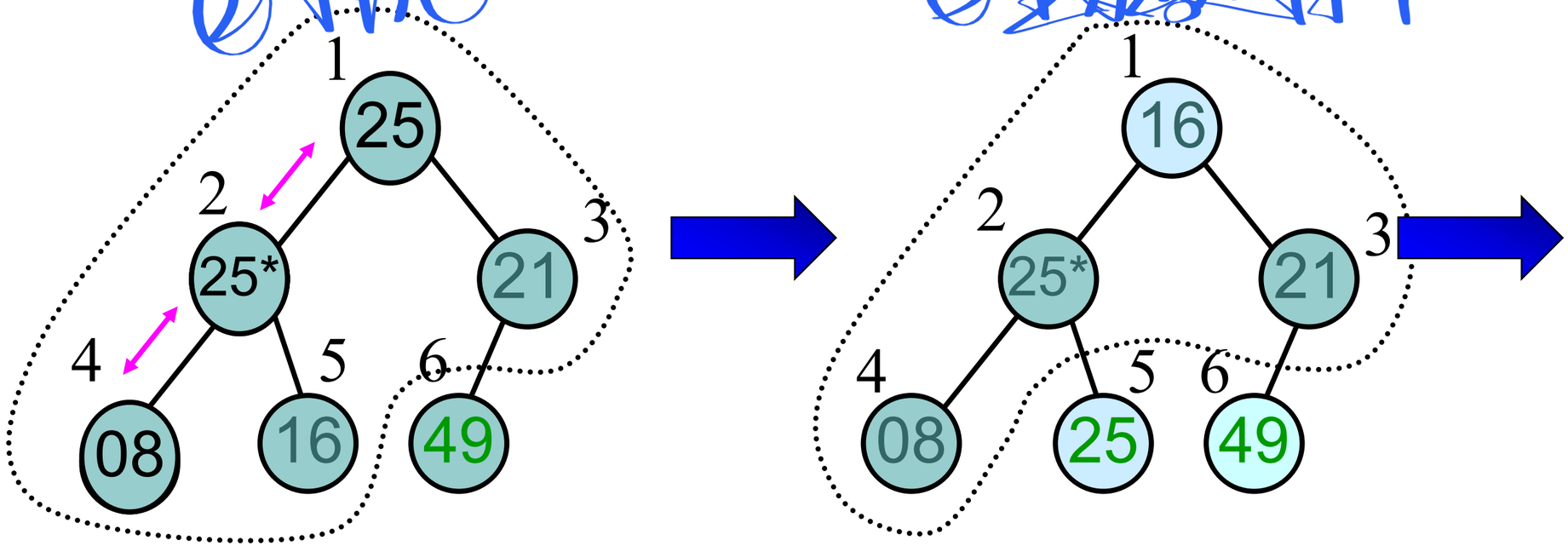
49	25	21	25*	16	08

初始堆

08	25	21	25*	16	49

交换 1号与 6 号记录

A Heapsort Example



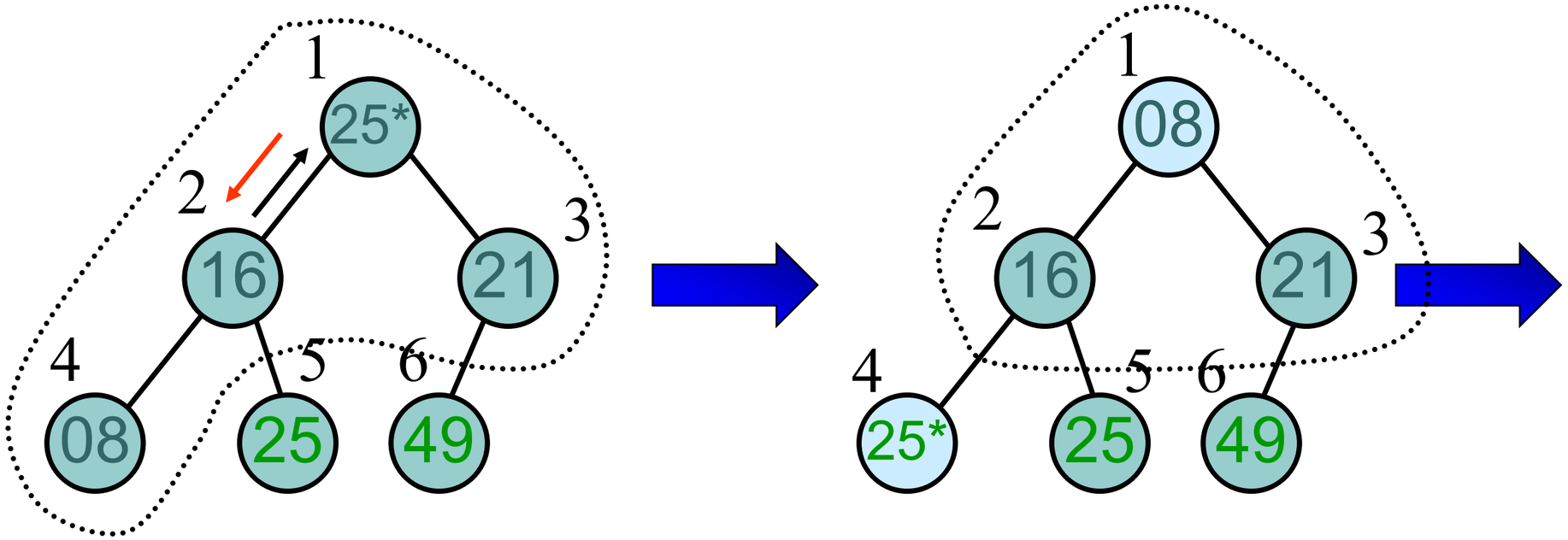
25	25*	21	08	16	49

从 1 号到 5 号 重新调整为堆

16	25*	21	08	25	49

交换 1号与 5 号记录

A Heapsort Example



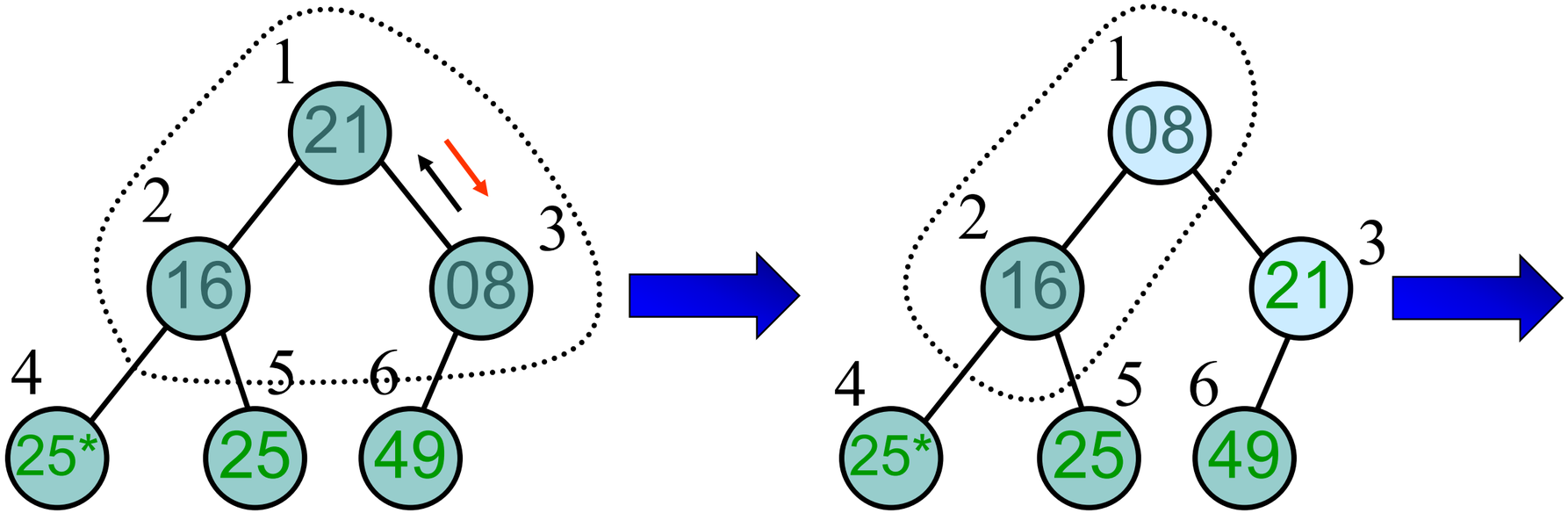
25*	16	21	08	25	49

从 1号到 4号 重新调整为堆

08	16	21	25*	25	49

交换 1 号与 4 号记录

A Heapsort Example



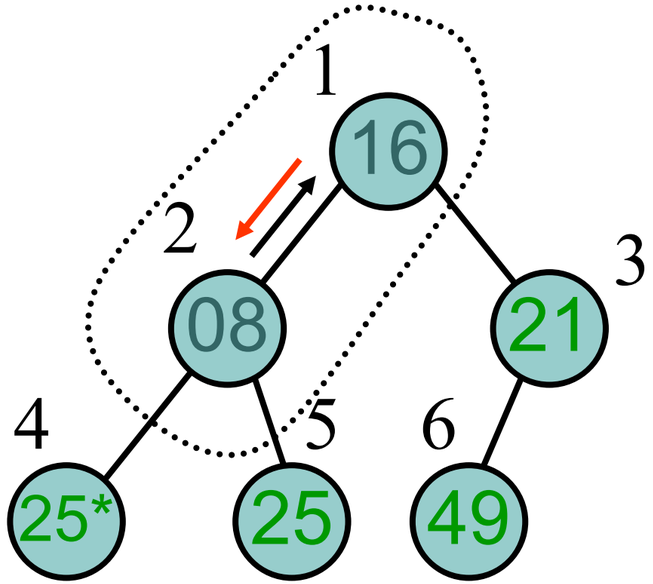
21	16	08	25*	25	49

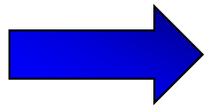
从 1 号到 3号 重新调整为堆

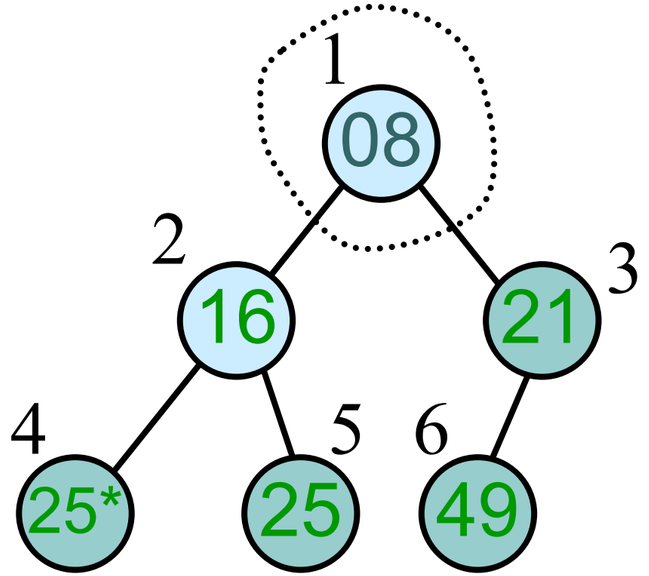
08	16	21	25*	25	49

交换 1号与3 号记录

A Heapsort Example







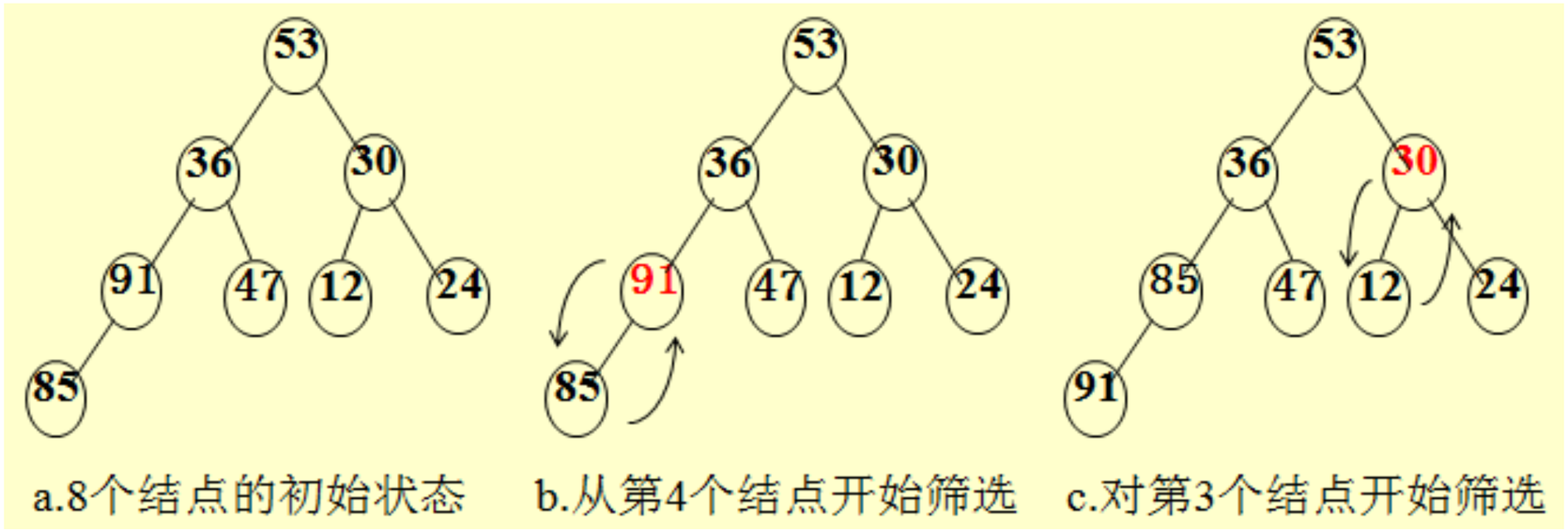
16	08	21	25*	25	49

从 1 号到 2 号 重新调整为堆

08	16	21	25*	25	49

交换 1 号与2 号记录

Building Heap Example



Heapsort Algorithm

堆排序

```
function heapsort(a, count) is
    input: an unordered array a of length count

    (Build the heap in array a so that largest value is at the root)
    heapify(a, count)

    (The following loop maintains the invariants that a[0:end] is a heap and every element
     beyond end is greater than everything before it (so a[end:count] is in sorted order))
    end ← count - 1
    while end > 0 do
        (a[0] is the root and largest value. The swap moves it in front of the sorted elements.)
        swap(a[end], a[0])
        (the heap size is reduced by one)
        end ← end - 1
        (the swap ruined the heap property, so restore it)
        siftDown(a, 0, end)
```

Heapsort Algorithm

建堆

```
(Put elements of a in heap order, in-place)
function heapify(a, count) is
    (start is assigned the index in a of the last parent node)
    (the last element in a 0-based array is at index count-1; find the parent of that element )
    start ← floor ((count - 2 ) / 2)

    while start ≥ 0 do
        (sift down the node at index start to the proper place such that all nodes below
         the start index are in heap order)
        siftDown(a, start, count-1)
        (go to the next parent node)
        start ← start - 1
    (after sifting down the root all nodes/elements are in heap order)
```

向下筛选

```
function siftDown(a, start, end) is
    root ← start

    while root * 2 + 1 ≤ end do      (While the root has at least one child)
        child ← root * 2 + 1         (left child)
        swap ← root                  (keeps track of child to swap with)

        if a[swap] < a[child]
            swap ← child
        (if there is a right child and that child is greater)
        if child+1 ≤ end and a[swap] < a[child+1]
            swap ← child + 1
        if swap ≠ root
            swap(a[root], a[swap])
            root ← swap              (repeat to continue sifting down the child now)
        else
            return
```

Analysis of Heapsort

- The "siftDown" version of heapify has *O(n)* time complexity.

> In its worst case for sorting a list of length n, heapsort performs
>
> $$2n \lg n + O(n)$$
>
> comparisons of keys, and it performs
>
> $$n \lg n + O(n)$$
>
> assignments of entries.

- Empirical studies show that (as for selection sort) there is relatively little difference between the average and worst cases, and heapsort usually takes about twice as long as quicksort.

Another Heapify Method

```
procedure heapify(a,count) is
    (end is assigned the index of the first (left) child of the root)
    end := 1

    while end < count
        (sift up the node at index end to the proper place such that all nodes above
         the end index are in heap order)
        siftUp(a, 0, end)
        end := end + 1
    (after sifting up the last node all nodes are in heap order)

procedure siftUp(a, start, end) is
    input:  start represents the limit of how far up the heap to sift.
                 end is the node to sift up.
    child := end
    while child > start
        parent := floor((child-1) / 2)
        if a[parent] < a[child] then (out of max-heap order)
            swap(a[parent], a[child])
            child := parent (repeat to continue sifting up the parent now)
        else
            return
```

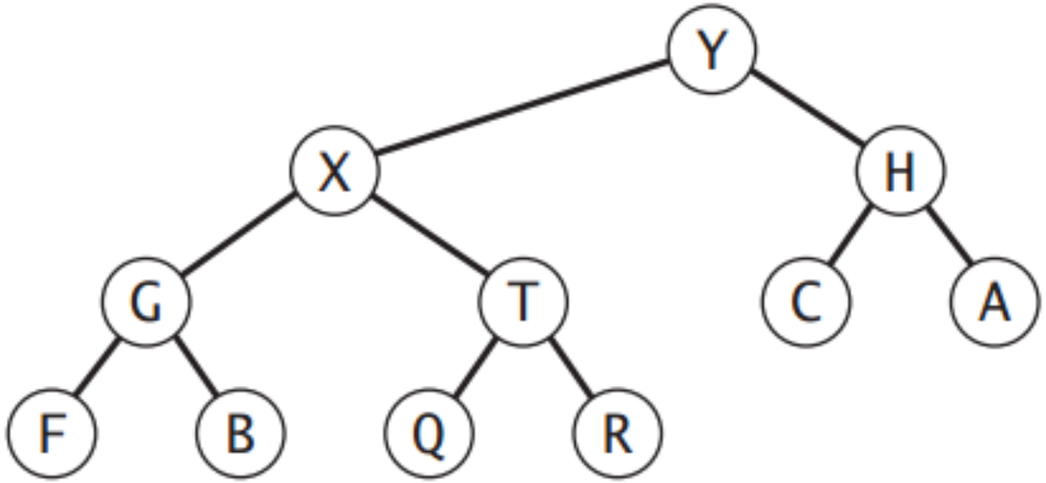
Exercise

- Consider the following binary tree representation of a max-heap.

  (a) Insert the key P into the heap

  (b) Remove the largest key from the heap

  Write down the resultant heap after each step using array representation.



Priority Queues

DEFINITION A ***priority queue*** consists of entries, such that each entry contains a key called the ***priority*** of the entry. A priority queue has only two operations other than the usual creation, clearing, size, full, and empty operations:

- Insert an entry.
- Remove the entry having the largest (or smallest) key.

Sift-up

Sift-down

If entries have equal keys, then any entry with the largest key may be removed first.

Thank you!

